

Introduction to Binary Trees

Definition

A binary tree is a hierarchical data structure characterized by nodes connected through edges, with each node having at most two children.

Key Terminology

Root, parent, child, leaf, subtree, depth, and height.

Purpose and Common Use Cases

Binary trees are used in various applications, including search algorithms, expression trees, and hierarchical data representation.

Characteristics of Binary Trees

- Each node has up to two direct child nodes.
- There is exactly one node, called the **root** of the tree, that has no parent node.
- All other nodes have exactly one parent.

Types of Binary Trees

1. Full Binary Trees

- **General Definition:** Every node has either 0 or 2 children.
- **Specific Detail:** A full binary tree is a special type of binary tree in which every parent node/internal node has either two or no children.

2. Complete Binary Trees

- **General Definition:** All levels are completely filled except possibly for the last level.
- **Key Rules:**
 1. Every level must be completely filled.
 2. All the leaf elements must lean towards the left.
 3. The last leaf element might not have a right sibling (i.e., a complete binary tree doesn't have to be a full binary tree).

3. Perfect Binary Trees

- **General Definition:** All internal nodes have exactly two children, and all leaf nodes are at the same level.
- **Specific Detail:** A perfect binary tree is a type of binary tree in which every internal node has exactly two child nodes and all the leaf nodes are at the same level.

Binary Tree Representation

Node Structure

Describes the structure of a typical binary tree node, containing data, a left child pointer, and a right child pointer.

Code Representation (C/C++)

C++

```
struct node
{
    int data;
    struct node *left;
    struct node *right;
};
```

Binary Tree Traversal Techniques

Tree traversal refers to the process of visiting all nodes in a tree data structure exactly once. There are three primary types of depth-first traversals:

1. Inorder Traversal

- **Logic:** Left Subtree $\rightarrow$ Root $\rightarrow$ Right Subtree.
- **Algorithm:**
 1. Traverse the left subtree (call `Inorder(left->subtree)`).
 2. Visit the root.
 3. Traverse the right subtree (call `Inorder(right->subtree)`).
- **Use Case:** In **Binary Search Trees (BST)**, an inorder traversal visits nodes in **non-decreasing (sorted) order**.

2. Preorder Traversal

- **Logic:** Root $\rightarrow$ Left Subtree $\rightarrow$ Right Subtree.
- **Algorithm:** Covers root, then visits left and right subtrees.

3. Postorder Traversal

- **Logic:** Left Subtree $\rightarrow$ Right Subtree $\rightarrow$ Root.
- **Algorithm:** Explores left and right subtrees before visiting the root.

C Implementation: Depth-First Traversals

C (Preorder)

```
void preorderTraversal(struct TreeNode* root) {  
    if (root == NULL) return;  
    printf("%d ", root->data);  
    preorderTraversal(root->left);  
    preorderTraversal(root->right);  
}
```

C (Postorder)

```
void postorderTraversal(struct TreeNode* root) {  
    if (root == NULL) return;  
    postorderTraversal(root->left);  
    postorderTraversal(root->right);  
    printf("%d ", root->data);  
}
```

- **Algorithm:** `Postorder(tree)`

Binary Tree Operations & Balanced Trees

Core Operations

- **Insertion:** The process of adding a new node to the tree.
- **Deletion:** The removal of a node from the tree.
- **Searching:** Searching for a specific value within the tree.

Balanced Binary Trees

A balanced binary tree (or height-balanced binary tree) is defined as a tree where the height of the left and right subtrees of any node differ by **not more than 1**.

Conditions for a height-balanced tree:

1. The difference between the left and right subtree heights for any node is ≤ 1 .
2. The left subtree is balanced.
3. The right subtree is balanced.

C Implementation: Checking Balance

C

```

int height(struct TreeNode* node) {
    if (node == NULL) return 0;
    int lh = height(node->left); int rh = height(node->right);
    return 1 + (lh > rh ? lh : rh);
}

int isBalanced(struct TreeNode* root) {
    if (root == NULL) return 1;
    int diff = abs(height(root->left) - height(root->right));
    return (diff <= 1 && isBalanced(root->left) && isBalanced(root->right));
}

```

Applications and Challenges

Binary Tree Applications

- **Data Storage:** Used in databases and file systems for efficient data storage and retrieval.
- **Expression Trees:** Used to represent and evaluate mathematical expressions.
- **Huffman Coding:** Used in data compression algorithms to assign variable-length codes to characters.

Challenges and Considerations

- **Handling Imbalanced Trees:** Imbalanced trees can lead to poor performance (shifting from $O(\log n)$ to $O(n)$ time complexity).
- **Memory Management:** Proper allocation and deallocation of nodes to prevent memory leaks.